

1. Graphics Hardware

Graphics hardware is responsible for generating and processing visual data. It includes:

a. GPU (Graphics Processing Unit)

- The brain of graphics hardware.
- Handles rendering of images, video, and animations.
- Modern GPUs (from NVIDIA, AMD, Intel) are also used for AI and computation.

b. Graphics Card (Video Card)

- A physical component that houses the GPU, memory (VRAM), and cooling system.
- Installed in desktops/laptops to boost visual performance.

c. VRAM (Video RAM)

- Dedicated memory in a graphics card.
- Stores textures, frame buffers, etc., for quick access by the GPU.

d. Display Interfaces

- **HDMI, DisplayPort, VGA, DVI, USB-C** – these are used to connect monitors and transfer video signals.
-

2. Display Technology

This includes the methods and devices used to present the visual output. Key types:

a. LCD (Liquid Crystal Display)

- Uses liquid crystals and backlighting.
- Common, affordable, and energy-efficient.

b. LED (Light Emitting Diode)

- A type of LCD that uses LEDs for backlighting.
- Brighter and more power-efficient than traditional LCDs.

c. OLED (Organic LED)

- Emits its own light (no backlight needed).

- Better contrast, thinner displays, and faster response times.

d. QLED (Quantum Dot LED)

- Enhanced LED with quantum dots for better color and brightness.

e. MicroLED

- Emerging tech similar to OLED but with better durability and brightness.

f. CRT (Cathode Ray Tube)

- Older technology using electron beams.
- Now obsolete, but historically important.

III Architecture of Raster-Scan Displays

1. Frame Buffer

- A dedicated block of memory that stores pixel data for the entire screen.
- Each pixel has data like color, intensity, and brightness.
- The display reads from this buffer to know what to show.

2. Display Controller

- Reads data from the frame buffer.
- Converts pixel data into signals that control the electron beam (in CRTs) or LEDs (in modern screens).
- Controls **timing** for horizontal and vertical scanning.

3. Raster Generator / Timing Generator

- Generates **synchronization signals**:
 - **Horizontal sync (H-sync)** – for each new line.
 - **Vertical sync (V-sync)** – for each new frame.
- Ensures the scan moves line-by-line and starts over correctly after each frame.

4. Digital-to-Analog Converter (DAC) (used in CRTs or analog displays)

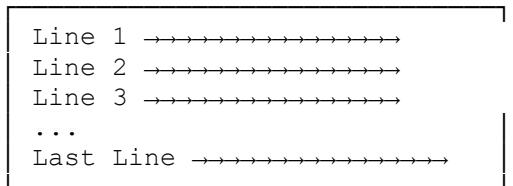
- Converts digital pixel data from the frame buffer into analog signals to drive the display.

5. Scanning System

- The electron beam (in CRT) or control signals (in modern displays) scan **left to right** and **top to bottom**.
 - The process is fast and repeated many times per second (e.g., 60Hz = 60 full frames per second).
-

📄 Raster Scan Working Principle

mathematica
CopyEdit
Start of Frame



End of Frame → Repeat (Vertical Retrace)

- **Horizontal Retrace:** When beam moves back to the start of the next line.
 - **Vertical Retrace:** When beam moves from bottom back to top to start a new frame.
-

✓ Key Characteristics

- **Progressive drawing** of the image line-by-line.
- Easy to implement and cheap.
- Common in TVs, monitors, projectors.

📄 Vector Displays (Also known as Calligraphic Displays)

Vector displays differ fundamentally from raster-scan displays. Instead of scanning every pixel row by row, **they draw images directly using lines**, much like how a plotter works. This was common in early **oscilloscopes**, **radar screens**, and **early computer graphics systems** (e.g., the Vectrex console or early flight simulators).

✂ Architecture of Vector Displays

1. Display Processor / Graphics Processor

- Receives instructions to draw lines (vectors).
- Each vector is defined by start and end coordinates (X_1, Y_1 to X_2, Y_2).

2. Digital-to-Analog Converters (DACs)

- Convert digital coordinate data into analog signals for deflecting the electron beam.

3. Beam Deflection System

- Controls the position of the electron beam using magnetic or electrostatic deflection.
- Moves the beam directly to the start point, turns it on, moves to the endpoint, then turns it off — drawing a **line**.

4. Electron Gun (CRT-specific)

- Shoots the electron beam at the screen to illuminate phosphor coating.
- Beam only turns on when drawing — no scanning of blank areas.

🔄 Working Principle

- Draws graphics by moving the beam from point to point.
- Only the required lines or shapes are drawn — **no background grid or pixel matrix**.
- **No refresh required** unless phosphor persistence fades (CRT dependent).

📁 Key Characteristics of Vector Displays

Feature	Description
Output Type	Lines (vectors) only
Image Quality	Very sharp lines, no pixelation
No Raster	Does not scan pixel rows
No Frame Buffer	Operates on instructions, not stored image data
Limited Fill Areas	Cannot display full-color images or shaded areas easily
Used in	Oscilloscopes, radar, early video games (like <i>Asteroids</i>), flight simulators

🔍 Vector vs Raster Display (Quick Comparison)

Feature	Raster Display	Vector Display
Drawing Method	Pixel grid (row by row)	Line-based (point to point)

Feature	Raster Display	Vector Display
Memory	Frame buffer needed	No frame buffer needed
Image Type	Can show full images, colors	Only line graphics
Common Use	Modern monitors/TVs	Oscilloscopes, legacy systems

Display Processors

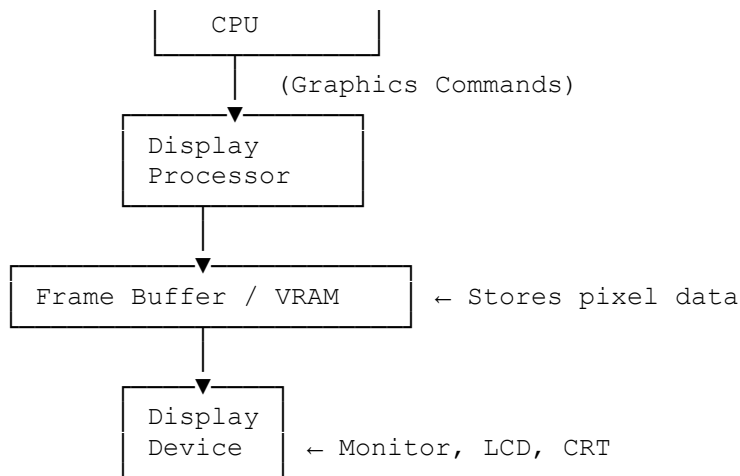
A **Display Processor** (also called a **Graphics Display Processor** or **Graphics Controller**) is a specialized component in graphics systems that manages the rendering and control of visual data before it is displayed on the screen. It acts as a **middle layer** between the **CPU** and the **display device**, offloading graphical tasks to improve system performance and display quality.

Functions of a Display Processor

- Graphics Command Execution**
 - Interprets drawing commands (e.g., line, circle, text).
 - Converts high-level instructions into displayable data.
 - Image Transformation**
 - Handles translation, rotation, scaling, and other geometric transformations.
 - Useful in 2D/3D rendering pipelines.
 - Clipping**
 - Removes or hides parts of graphics outside the viewing area.
 - Rasterization**
 - Converts vector data (lines, shapes) into pixel data (raster format) for display.
 - Shading and Lighting Calculations**
 - (In 3D systems) Calculates surface lighting, shading, and colors.
 - Buffer Management**
 - Manages frame buffers, z-buffers, and color buffers used during rendering.
 - Refresh and Timing Control**
 - Manages the timing of frame updates and synchronization with the display (horizontal and vertical retrace).
 - Video Signal Generation**
 - In simpler systems, it generates the actual signals to drive CRT or LCD panels.
-

Architecture of a Display Processor (Simplified)

mathematica
CopyEdit



❑ Examples of Display Processors

Processor Type	Description
GPU (Graphics Processing Unit)	Modern display processors with high power, used for 3D rendering, gaming, and AI
Display Controller (e.g., TCON)	Manages screen timing and refresh, often in embedded/mobile devices
Raster Display Processor	Handles rasterization and output for raster-scan displays
Vector Display Processor	Generates analog signals to draw vectors on screen

✓ Advantages

- Offloads work from the CPU
- Enables real-time rendering
- Improves overall system performance and display quality

💠 Hard Copy Devices

Definition:

A **hard copy device** is a device that produces a **physical (permanent) output** of data stored or processed in a computer — usually in the form of **paper, film, or printed media**.

➡ Examples of Hard Copy Devices:

Device	Description
Printer	Converts digital documents into printed text/images on paper.
Plotter	Used for printing large graphics like engineering drawings (CAD), blueprints, etc.
Typewriter Terminals	Older devices that typed digital input directly onto paper.
Fax Machine	Sends and prints documents over a phone line.
Photographic Film Recorders	Used to output high-resolution images onto film.

◆ Input Devices

Definition:

An **input device** is any hardware component that allows **users to enter data or control signals** into a computer.

Examples of Input Devices:

Device	Function
Keyboard	Inputs text and commands via key presses.
Mouse	Points, clicks, drags objects on the screen.
Scanner	Digitizes physical documents or images.
Microphone	Captures audio input.
Camera/Webcam	Captures image/video input.
Joystick/Gamepad	Inputs movement and commands in games.
Touchscreen	Combines display and input by detecting touch.
Stylus/Digital Pen	Inputs handwriting or drawings on tablets.
Biometric Devices	Inputs identity via fingerprint, face, retina, etc.

Comparison Table

Feature	Hard Copy Device	Input Device
Purpose	Output physical documents	Input data into the system
Direction	Output	Input
Examples	Printer, plotter	Keyboard, mouse, scanner
Data Type	Text, image, graphics	Text, image, audio, gesture

Graphics Software

Graphics software includes programs, libraries, and tools used to create, render, manipulate, and display graphical content on computers. It interacts closely with graphics hardware and follows **standards** and **languages** to ensure portability and performance.

1. Software Standards in Graphics

Graphics software standards are sets of rules or protocols that ensure graphics applications behave consistently across different platforms and devices. These standards help developers write graphics programs that are **portable**, **efficient**, and **hardware-independent**.

✓ Common Graphics Software Standards:

Standard	Description
OpenGL (Open Graphics Library)	Cross-platform API for 2D/3D rendering. Widely used in CAD, gaming, and simulations.
DirectX	A Microsoft-specific API suite for multimedia and gaming on Windows.
Vulkan	Low-overhead, high-performance API for 3D graphics and compute tasks.
WebGL	JavaScript API for rendering interactive 3D graphics within web browsers.
GKS (Graphical Kernel System)	Early standard for 2D vector graphics; ISO-standardized.

Standard	Description
PHIGS (Programmer's Hierarchical Interactive Graphics System)	ISO standard for 3D graphics; structured and hierarchical.

💡 2. Need for a Machine-Independent Graphics Language

❑ Why Machine Independence Matters:

- Portability**
 - Code written once should work on different hardware or operating systems.
 - Essential for software that runs across desktops, mobile, and web platforms.
 - Hardware Abstraction**
 - Developers shouldn't need to understand GPU internals or hardware details.
 - The graphics language (API) handles communication with the hardware.
 - Cross-Platform Development**
 - Reduces development time and cost by avoiding platform-specific codebases.
 - Ease of Learning & Maintenance**
 - Common, standard APIs are easier to learn and maintain across teams.
 - Compatibility with Various Devices**
 - From high-end desktops to embedded systems and smartphones.
-

🌐 Examples of Machine-Independent Graphics Languages

Language/API	Machine-Independent?	Notes
OpenGL	✓ Yes	Works on Windows, macOS, Linux
WebGL	✓ Yes	Runs in web browsers
DirectX	✗ No (Windows-only)	Limited to Microsoft platforms
Vulkan	✓ Yes	More control, high performance

★ Summary

- Graphics software standards** ensure uniform behavior across systems.
- A **machine-independent graphics language** is vital for cross-platform, efficient, and maintainable software.
- APIs like **OpenGL**, **WebGL**, and **Vulkan** are essential tools for modern graphics development.

✦ Scan Converting a Point and a Straight Line

Scan conversion is the process of converting **geometric primitives** (like points, lines, circles) into a set of **pixels** on a raster display.

- **Point scan conversion** is simple: plot the pixel at (x, y) directly.
- **Line scan conversion** is more complex and uses algorithms like:
 - **DDA (Digital Differential Analyzer) Line Algorithm**
 - **Bresenham's Line Algorithm**

1. DDA Line Algorithm (Digital Differential Analyzer)

✓ Concept:

- Uses **floating-point arithmetic**.
- Calculates intermediate points between the start and end using the **slope** of the line.

□ Steps:

1. Calculate:
 - $dx = x_2 - x_1, dy = y_2 - y_1$
 - $steps = \max(|dx|, |dy|)$
2. Find increments:
 - $x_inc = dx / steps$
 - $y_inc = dy / steps$
3. Start from (x_1, y_1) and plot $steps$ points by adding x_inc and y_inc repeatedly.

✎ Pseudo Code:

```
c
CopyEdit
x = x1; y = y1;
for (int i = 0; i < steps; i++) {
    plot(round(x), round(y));
    x += x_inc;
    y += y_inc;
}
```

✓ Pros:

- Simple to implement

✗ Cons:

- Uses floating-point calculations → slower on hardware without FPUs

2. Bresenham's Line Algorithm

✓ Concept:

- Uses **only integer arithmetic**.
- Decides pixel positions based on **error terms** between the ideal line and the rasterized one.

□ Steps (for gentle slopes, i.e., $|\text{slope}| < 1$):

1. Calculate:
 - $dx = x_2 - x_1, dy = y_2 - y_1$
 - $p = 2*dy - dx$ (initial decision parameter)
2. Loop through x :
 - If $p < 0$: next pixel is $(x+1, y)$, update $p = p + 2*dy$
 - If $p \geq 0$: next pixel is $(x+1, y+1)$, update $p = p + 2*dy - 2*dx$

📋 Pseudo Code (simplified):

```
c
CopyEdit
p = 2*dy - dx;
x = x1; y = y1;
for (int i = 0; i <= dx; i++) {
    plot(x, y);
    x++;
    if (p < 0) {
        p += 2*dy;
    } else {
        y++;
        p += 2*(dy - dx);
    }
}
```

✓ Pros:

- Very efficient (integer-only arithmetic)
- Used in hardware and embedded systems

✗ Cons:

- More complex logic, especially for steep or negative slopes
-

Comparison: DDA vs. Bresenham

Feature	DDA Algorithm	Bresenham's Algorithm
Arithmetic	Floating-point	Integer-only
Speed	Slower	Faster
Accuracy	Good	Better (no rounding errors)
Complexity	Simple	Slightly complex
Use in Hardware	Rare	Common

Scan Conversion of Circle and Ellipse

When converting **circles** and **ellipses** to raster format, we use **mid-point algorithms** to decide which pixels to turn on, using **integer arithmetic** for efficiency.

1. Mid-Point Circle Algorithm

Concept:

- Based on symmetry: a circle is symmetric about its center in all 8 octants.
- Only one **octant** is calculated; the rest are mirrored.
- Decision parameter determines whether the **next pixel** is **E (East)** or **SE (South-East)**.

Circle Equation:

For a circle centered at $(0, 0)$:

$$x^2 + y^2 = r^2$$

Steps:

1. Start at point $(x = 0, y = r)$
2. Initial decision parameter:

$$p_0 = 1 - r$$

3. For each x , decide:
 - If $p < 0$: next point is $(x+1, y)$

$$p_{\text{next}} = p + 2x + 3 \quad p_{\text{next}} = p + 2x + 3$$

- If $p \geq 0$: next point is $(x+1, y-1)$

$$p_{\text{next}} = p + 2x - 2y + 5 \quad p_{\text{next}} = p + 2x - 2y + 5$$

4. Plot 8 symmetric points for each (x, y) .

↻ Symmetry (Octants):

If (x, y) is on the circle:

- Plot: $(\pm x, \pm y), (\pm y, \pm x)$

2. Mid-Point Ellipse Algorithm

✓ Concept:

- Uses the ellipse equation and symmetry across **four quadrants**.
- Divides the first quadrant into two regions:
 - **Region 1**: slope ≤ 1
 - **Region 2**: slope > 1
- Uses separate decision parameters in each region.

▴ Ellipse Equation:

Centered at $(0, 0)$:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1 \quad \frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

□ Steps:

Region 1 (Initial part, slope ≤ 1):

- Start at $(0, b)$
- Decision parameter:

$$p_1 = b^2 - a^2b + \frac{1}{4}a^2 \quad p_1 = b^2 - a^2b + \frac{1}{4}a^2$$

- If $p < 0$: next point is $(x+1, y)$
- Else: $(x+1, y-1)$

Region 2 (After slope > 1):

- Switch to using different logic for steeper slope
- Decision parameter:

$$p_2 = b^2(x+0.5)^2 + a^2(y-1)^2 - a^2b^2p_1 = b^2(x+0.5)^2 + a^2(y-1)^2 - a^2b^2p_1$$

4-way symmetry: Plot $(\pm x, \pm y)$ for every calculated point.

Comparison Table: Mid-Point Circle vs Ellipse Algorithm

Feature	Circle Algorithm	Ellipse Algorithm
Equation Used	$x^2 + y^2 = r^2$	$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$
Symmetry Used	8-way	4-way
Regions to Divide	1 region	2 regions
Complexity	Simple	More complex
Efficiency	High (integer math)	High

Area Filling Algorithms in Computer Graphics

Area filling refers to the process of coloring or shading the **interior region** of a closed shape or boundary on a raster display.

1. Scan-Line Polygon Fill Algorithm

✓ Concept:

- Fills polygons by scanning each horizontal **scan line** across the image and determining **intersection points** with polygon edges.
- Pairs of intersections are filled between left and right boundaries.

🔧 Steps:

1. For each scan line:
 - Find all polygon edge intersections with the scan line.

- Sort intersections by increasing x.
 - Fill pixels between pairs of x-values (x1 to x2, x3 to x4, etc.).
2. Repeat for all scan lines intersecting the polygon.

✓ Advantages:

- Efficient for **convex and concave** polygons.
- Works well in raster-based systems.

● 2. Inside-Outside Test (Odd-Even Rule / Winding Number Rule)

◆ Odd-Even Rule:

- Draw a **ray** from the point to the right.
- **Count** how many times it intersects polygon edges.
- If the count is **odd**, the point is **inside**; if **even**, it's **outside**.

◆ Winding Number Rule:

- Measures how many times the polygon winds around the point.
- Positive (non-zero): inside
- Zero: outside
- Useful for **complex** or **self-intersecting** polygons.

📐 3. Scan-Line Fill of Curved Boundary Areas

- For curved shapes (like ellipses or Bézier curves), the scan-line method can still be used:
 1. **Compute curve intersections** with each scan line (mathematically).
 2. Fill between intersection points as with polygons.
- Needs **precise curve equations** (e.g., parametric or i) and accurate intersection computation.

● 4. Boundary-Fill Algorithm

✓ Concept:

- Start from a **seed point** inside the boundary.
- Recursively fill neighboring pixels until the **boundary color** is reached.

☐ Types:

- **4-connected**: fills up/down/left/right neighbors.

- **8-connected:** includes diagonals as well.

🔧 Algorithm:

```
pseudo
CopyEdit
function boundaryFill(x, y, fillColor, boundaryColor):
    if pixel(x, y) ≠ boundaryColor AND pixel(x, y) ≠ fillColor:
        set pixel(x, y) = fillColor
        call boundaryFill for all neighbors
```

✗ Disadvantages:

- Can be slow or cause **stack overflow** with large areas.
- Not ideal for complex shapes or raster gaps.

🔗 5. Flood-Fill Algorithm

✓ Concept:

- Similar to boundary fill, but instead of stopping at a boundary color, it stops at any **different color** than the starting color.
- Used in paint programs (e.g., bucket tool).

🔧 Algorithm:

```
pseudo
CopyEdit
function floodFill(x, y, targetColor, fillColor):
    if pixel(x, y) == targetColor:
        set pixel(x, y) = fillColor
        call floodFill for all neighbors
```

🔄 Use Case Difference:

Feature	Boundary Fill	Flood Fill
Stops At	A defined boundary color	Any color different from target
Used In	Closed shape filling	Free region or area fill

✓ Summary Table

Algorithm	Works With	Uses Recursion?	Notes
Scan-Line Polygon Fill	Polygons	No	Efficient and fast
Inside-Outside Test	Point location in shape	No	For determining “in/out” status
Curved Scan-Line Fill	Curved areas	No	Needs curve intersections
Boundary Fill	Enclosed shapes	Yes	Stops at boundary color
Flood Fill	Connected regions	Yes	Stops at color change

📐 Unit 3: Two-Dimensional Geometric Transformations

In this unit, we will explore the basic **2D transformations** and the concepts around coordinate transformations used in graphics, particularly in the **Viewing Pipeline** and **Window to Viewport Transformation**.

3.1 Two-Dimensional Transformations:

◆ Translation

Translation involves shifting an object from one position to another without altering its shape or orientation.

- **Formula:**
 - For a point (x, y) translated by (t_x, t_y) :

$$(x', y') = (x + t_x, y + t_y) \quad (x', y') = (x + t_x, y + t_y)$$

- **Transformation Matrix (2D):**

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

◆ Rotation

Rotation turns an object around a fixed point, usually the origin. The angle is measured counterclockwise.

- **Formula** (Rotation by angle θ):

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

◆ Scaling

Scaling changes the size of an object, either enlarging or shrinking it. It can be uniform (same factor in both x and y) or non-uniform (different scaling factors for x and y).

- **Formula:**
 - For scaling factors S_x and S_y :

$$(x', y') = (x \cdot S_x, y \cdot S_y) \quad (x', y') = (x \cdot S_x, y \cdot S_y)$$

- **Transformation Matrix:**

$$\begin{pmatrix} S_x & 0 \\ 0 & S_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} S_x \cdot x \\ S_y \cdot y \end{pmatrix}$$

◆ Reflection

Reflection mirrors an object across a line (e.g., the x-axis or y-axis) or about a point.

- **Reflection about x-axis:**

$$(x', y') = (x, -y) \quad (x', y') = (x, -y)$$

- **Reflection about y-axis:**

$$(x', y') = (-x, y) \quad (x', y') = (-x, y)$$

- **Reflection Matrix (about x-axis):**

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x \\ -y \end{pmatrix}$$

◆ Shearing

Shearing shifts the position of an object in a direction parallel to one of its axes, distorting its shape.

- **Shear along x-axis:**

$$(x', y') = (x + k \cdot y, y) \quad (x', y') = (x + k \cdot y, y)$$

- **Shear along y-axis:**

$$(x', y') = (x, y + k \cdot x) \quad (x', y') = (x, y + k \cdot x)$$

- **Shear Matrix (along x-axis):**

$$\begin{pmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

3.2 Homogeneous Coordinates and 2D Composite Transformations

◆ Homogeneous Coordinates

To simplify the combination of multiple transformations, we use **homogeneous coordinates**, where a point (x, y) in 2D is represented as $(x, y, 1)$ in 3D coordinates. This allows transformations like translation, which cannot be represented in standard 2D coordinates, to be done in matrix form.

- **Homogeneous Representation:**

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- **Translation Matrix in Homogeneous Coordinates:**

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

◆ Composite Transformations

Composite transformations involve performing multiple transformations (e.g., rotation + scaling) in a single operation by multiplying transformation matrices.

- **Example:**
 - Applying translation, followed by scaling, involves multiplying the translation matrix by the scaling matrix:

$$T \cdot S = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

3.3 Two-Dimensional Viewing: Viewing Pipeline, Window to Viewport Coordinate Transformation

◆ Viewing Pipeline

The **viewing pipeline** refers to the process of transforming 3D world coordinates into 2D screen coordinates. The process involves several steps:

1. **World Coordinate Transformation:**
 - Converts world coordinates to eye coordinates (view transformation).
2. **Projection Transformation:**
 - Converts eye coordinates to normalized device coordinates (perspective or orthographic projection).
3. **Viewport Transformation:**
 - Maps normalized device coordinates to screen coordinates.

◆ Window to Viewport Coordinate Transformation

This transformation maps a **window** (a rectangular section of the world coordinate system) to a **viewport** (a rectangular section of the screen or window).

- **Window:** Defined by min/max coordinates $(x_w, y_w)(x_w, y_w)$ and $(x_w', y_w')(x_w', y_w')$.
- **Viewport:** Defined by screen coordinates $(x_v, y_v)(x_v, y_v)$ and $(x_v', y_v')(x_v', y_v')$.

□ Window to Viewport Transformation Formula:

For each point $(x_w, y_w)(x_w, y_w)$ in the window:

$$x_v = \frac{(x_w - x_{w_min})}{(x_{w'} - x_{w_min})} (x_v' - x_v) + x_v \quad y_v = \frac{(y_w - y_{w_min})}{(y_{w'} - y_{w_min})} (y_v' - y_v) + y_v$$

This scales and translates the window to fit the viewport.

✓ Summary of Transformations

Transformation	Matrix (2D)	Description
Translation	$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$	Moves points by t_x, t_y
Rotation	$\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$	Rotates by angle θ
Scaling	$\begin{pmatrix} S_x & 0 \\ 0 & S_y \end{pmatrix}$	Scales by factors S_x, S_y

Transformation	Matrix (2D)	Description
Reflection	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ (about x-axis)	Reflects across axes
Shearing	$\begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$	Distorts by factor k
Homogeneous Coordinates	$\begin{pmatrix} x & y & 1 \end{pmatrix}$	Extended for transformations in 3D space